PERFORMANCE TESTING REPORT

Hostel Management System

Quaid-i-Azam University
Software Testing Documentation

Document status: Prepared test plan and test cases; execution results pending
Version: 1.0 | Date: 26 October 2026

Document Control

Field	Details
Document title	Performance Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Planned test cases documented; empirical measurements not supplied
Repository destination	reports/performance-testing-report.docx on the report branch

1. Introduction

This report defines the performance-testing approach for the Hostel Management System (HMS) developed for Quaid-i-Azam University. Performance testing evaluates how the system responds to user actions and behaves when multiple users access key functions at the same time. The focus is on common student and staff workflows, database-dependent operations, and report generation.

2. Objectives

- Assess response time for common screens and user actions.
- Check whether important workflows remain usable as concurrent user activity increases.
- Observe throughput, error rate, and resource usage during representative workloads.
- Identify slow database queries, inefficient filtering, and expensive report-generation operations.
- Record reproducible measurements so future changes can be compared against a baseline.

3. Scope

Included area	Performance activity
Authentication and dashboards	Open login, authenticate a test account, and load the relevant dashboard.
Hostel applications	Submit an application and retrieve or filter application records.
Room and bed allotment	Search available beds and save an allotment using test data.
Complaint management	Submit a complaint and retrieve complaint lists/statuses.
Fee management	Retrieve fee status and record a test payment verification where supported.
Visitor management	Record visitor entry/exit and search visitor records.
Notices	Load the notice list and student-facing notices.
Reports	Generate occupancy and other available administrative reports.

Out of scope: security penetration testing, functional correctness beyond checks needed to validate test transactions, and production capacity certification. Those require separate testing and deployment information.

4. Test Environment and Preconditions

The actual implementation stack, server specifications, database engine, hosting configuration, and network conditions must be recorded before execution. These details were not provided with this report, so they are intentionally left as fields to complete rather than assumed.

Environment item	Value to record before testing
Application build / commit	_____
Application URL / deployment type	_____
Operating system and machine specifications	_____
Database engine and version	_____
Test tool and version	_____
Network conditions	_____

Test-data volume (students, applications, beds, etc.)	_____
Test date and tester	_____

Preconditions:

- Use a non-production test environment and test accounts for student and authorized staff roles.
- Prepare representative data for students, applications, hostels, rooms, beds, complaints, payments, visitors, and notices.
- Confirm that database backups or reset procedures are available before tests that create or update records.
- Ensure the test workload does not target a live university service without permission.
- Synchronize clocks and capture application, database, and load-generator logs where available.

5. Performance Metrics

Metric	Meaning	How to record
Response time	Time from request/action start until the response is usable.	Median, 90th percentile (p90), and 95th percentile (p95).
Throughput	Completed requests or transactions per second/minute.	Report by scenario and overall.
Error rate	Failed requests, timeouts, or unexpected server errors divided by total requests.	Percentage plus error categories.
Concurrent users	Number of simulated users active at the same time.	Record configured and achieved concurrency.
Resource usage	CPU, memory, database connections, and other available resource indicators.	Capture peak and average during each run.
Data integrity	Whether load caused duplicate or missing records or invalid state changes.	Validate records after the run.

6. Workload Model

Use the following as a proposed staged workload, not as a claim about the system's expected real-world capacity. Adjust the levels after discussing expected usage with the project team and confirming available test hardware.

Stage	Proposed concurrent users	Duration	Purpose
Baseline	1–5	5 minutes	Measure basic response times and validate the test setup.
Normal load	10	10 minutes	Observe common student and staff activity.
Increased load	25	10 minutes	Check response-time trends as concurrency increases.
Higher load	50	10 minutes	Explore behavior under a heavier workload, if the environment permits.
Spike test	Increase rapidly from 5 to 25	5 minutes	Observe recovery and error behavior after a sudden increase.
Soak test	10	30–60 minutes	Look for gradual degradation, resource leaks, or rising error rates.

The proposed user counts and durations are initial test settings only. They are not service-level commitments, and should be scaled to the intended deployment and available resources.

7. Performance Test Cases

ID	Scenario	Procedure / workload	Expected result	Status
PT-01	Login page load	Open the login page repeatedly with a clean test session.	Page loads successfully; record median, p90, p95 and errors.	Not run
PT-02	Login under concurrency	Run valid logins for authorized test accounts at the selected user level.	Authentication remains correct; record latency and error rate.	Not run
PT-03	Dashboard load	Load student and staff dashboards with representative data.	Correct summary data appears; record response times and failures.	Not run
PT-04	Student record search	Search for existing student records using representative search terms.	Correct records are returned; latency is recorded.	Not run
PT-05	Application filtering	Filter applications by status and other available filters.	Filtered results are correct and the response is measured.	Not run

PT-06	Submit hostel application	Submit valid applications using unique test accounts/data.	Confirmation is returned and each valid application is stored once.	Not run
PT-07	Bed availability search	Search available rooms/beds while staff accounts perform searches concurrently.	Availability results are returned; no incorrect availability is introduced.	Not run
PT-08	Bed allotment	Create allotments for distinct eligible students and beds.	Valid allotments are saved; no bed or student receives conflicting active allotments.	Not run
PT-09	Complaint submission/list	Submit test complaints and load the complaint list.	Requests complete correctly; record latency and errors.	Not run
PT-10	Fee status / verification	Retrieve fee status and verify test payments if the feature is implemented.	Records remain consistent and the response is measured.	Not run
PT-11	Visitor records	Create test visitor entries and search or update visitor records.	Visitor records are correctly saved and retrieved.	Not run
PT-12	Notice list	Load the student notice list with representative notice data.	Notices display correctly and response time is recorded.	Not run
PT-13	Occupancy report	Generate an occupancy report for a representative hostel/data range.	Report totals match database records; generation time is recorded.	Not run
PT-14	Mixed workload	Run dashboard, search, application, complaint, and report scenarios together.	System remains stable; throughput, latency, errors, and resource use are captured.	Not run
PT-15	Spike and recovery	Increase simulated users rapidly, then return to the normal workload.	Record timeouts/errors during the spike and whether normal performance recovers.	Not run
PT-16	Soak test	Run a sustained representative workload for 30–60 minutes.	No unexplained degradation, growing errors, or resource exhaustion is observed.	Not run

8. Acceptance Criteria

The project documents do not define numeric service-level targets, and no deployment benchmark was supplied. Therefore, the following are proposed criteria to confirm with the project supervisor before testing:

- Core workflows complete without unexpected failures under the agreed normal workload.
- Response-time targets are agreed and written down before the final run; report p50, p90, and p95 against those targets.
- No duplicate active bed allotments or duplicate application submissions are introduced by concurrent requests.
- Occupancy report totals and other checked outputs remain consistent with stored test data.
- Errors, timeouts, and resource consumption remain within thresholds agreed for the test environment.
- After spike testing, the application returns to the normal workload without requiring an unplanned restart.

Do not mark the system as passing solely because pages appear to load. A final pass/fail decision requires measured results and agreed numeric targets.

9. Results and Measurements

This section is intentionally unpopulated with performance numbers because no load-test output or measured execution data was supplied. Fill it after running the tests.

Scenario / ID	Users	Requests / transactions	Median (ms)	p95 (ms)	Error rate	Outcome
Baseline / PT-01	—	—	—	—	—	Pending
Normal load / PT-02–PT-13	—	—	—	—	—	Pending
Increased load / PT-14	—	—	—	—	—	Pending
Spike / PT-15	—	—	—	—	—	Pending
Soak / PT-16	—	—	—	—	—	Pending

9.1 Defect and Observation Log

ID	Scenario	Observed issue / evidence	Severity	Action / owner	Status
OBS-01	To be completed	Record measured bottlenecks, errors, or unexpected behavior.	TBD	TBD	Open
OBS-02	To be completed	Add a second row for each distinct finding.	TBD	TBD	Open

10. Recommended Execution Procedure

1. Confirm the environment, version/commit, test data volume, and approved workload.
2. Choose a load-testing tool compatible with the application's actual protocol and architecture.
3. Validate each scenario with one user before adding concurrency.
4. Run the baseline and record latency percentiles, throughput, errors, and resource use.
5. Execute the staged workload levels, keeping a separate result record for each run.
6. Check logs and database state after write-heavy scenarios; confirm there are no duplicate or inconsistent records.
7. Repeat anomalous runs to distinguish reproducible defects from temporary environment effects.
8. Attach test-tool exports, charts, timestamps, and relevant logs to the project evidence folder.
9. Update the results and defect sections, then assign pass/fail only against agreed criteria.

11. Risks and Limitations

- The deployed application and its technology stack have not been identified in this report.
- The suggested concurrent-user counts are planning values, not a verified capacity estimate.
- Network latency, machine load, database size, and test-data quality can affect results.
- Load tests against production may disrupt real users; use an authorized test environment.
- A performance run does not replace functional, integration, security, or user-acceptance testing.

12. Conclusion

This report provides a project-specific performance-testing plan for the Hostel Management System, covering its principal student and staff workflows and the measurements needed to evaluate responsiveness, concurrency, stability, and data consistency. The report is ready to be added to the project's report branch as a planned test document. Final performance conclusions must be added after execution using actual tool output and the agreed acceptance criteria.

Appendix A — Evidence Checklist

☐ Application build/commit and environment details

☐ Load-test scenario/configuration export

☐ Results for baseline, normal, increased, spike, and soak workloads

☐ Latency percentile and throughput charts

☐ Error summary and relevant application/database logs

☐ Post-run data-integrity checks

☐ Defect tickets and retest evidence, if any

Appendix B — Suggested Repository Placement

File: reports/performance-testing-report.docx

Store supporting screenshots, raw result exports, and graphs in a clearly named evidence folder if your repository structure supports it, for example: reports/evidence/performance/.